

COMP3334 Project Report

Team 62

Name	Student ID	Contribution(%)
Inha Pang	22096862D	28
Sun chang	22100709D	16
YUNG Kwok Tung	22041871D	28
WANG Yihe	21099191D	28

Abstract

Our application presents a secure cloud storage system based on client-server architecture. The system protects user data through end-to-end encryption and strict access control to protect user data.

We successfully address all core functionality, such as user management, data encryption, and access control. In addition, we have incorporated extended features such as Multi-Factor Authentication (MFA) to further strengthen the system's capabilities.

For technical implementation, we chose Python for its versatility and SQLite for lightweight database management. The combination delivers both security and usability.

Introduction

We present a secure cloud storage system that solves critical data protection challenges. Our solution combines strong security with practical utility.

Background

Cloud storage usage continues growing rapidly. However, security incidents frequently expose sensitive data. Existing solutions often force users to choose between protection and convenience. Our work addresses this gap by implementing essential security features without compromising accessibility.

Threat models

A threat model is a structured representation of all information that affects the security of an application, including potential threats and mitigations [1].

Two main adversaries

- Server-Side Passive Adversary: Operates the storage service correctly but attempts to decrypt user files. This adversary can monitor all encrypted data and network communications but cannot alter system behavior or modify transmitted content.
- Unauthorized User Adversary: Gains access to a legitimate user's device. May use a legitimate user's computer to try to access the online files of that legitimate user.

Both adversaries are limited to passive attacks: they cannot perform active manipulations like altering data or injecting malicious code.

Algorithms and Implementation Details → Server Side

User Management

- Theoretical Design:
 - a. Registration: Users provide a username and password. The server hashes the password and stores it securely.
 - b. Login: The server verifies the password against the stored hash and issues a JWT token.
 - c. Password Reset: A secure token is generated, stored hashed, and sent to the user for verification.
- Implementation:

The system securely hashes passwords with bcrypt before storage, never keeping plaintext versions. This standard security practice protects credentials even if the database is compromised.

 - a. Registration: Firstly, use bcrypt to hash the password before storing it in the database (user table). Then ensure the plaintext passwords are never stored.

```
# Register endpoint - handles new user creation
@app.route("/register", methods=["POST"])
def register():
    # Hash password with bcrypt using generated salt
    password_hash = bcrypt.hashpw(data["password"].encode(), bcrypt.gensalt())

    # Generate TOTP secret for multi-factor authentication
    totp_secret = pyotp.random_base32()

    try:
        # Insert new user with hashed credentials into database
        # SQL UNIQUE constraint ensures username uniqueness
        c.execute("INSERT INTO users (username, password_hash, public_key, totp_secret) VALUES (?, ?, ?, ?)",
                  (username, password_hash, public_key, totp_secret))
    except sqlite3.IntegrityError:
        # Handle duplicate username error
        return jsonify({"error": "Username taken"}), 400
```

- b. Login: Compares the submitted password with the stored hash using `bcrypt.checkpw()`. Issues a JWT token for session management.

```
# Login endpoint - authenticates existing users
@app.route("/login", methods=["POST"])
def login():
    # Retrieve user credentials from database
    c.execute("SELECT user_id, password_hash, totp_secret FROM users WHERE username = ?", (data["username"],))

    # Verify password against stored hash
    if user and bcrypt.checkpw(data["password"].encode(), user[1]):
        if user[2]: # Check if MFA is enabled
            # Generate temporary token for MFA verification flow
            temp_token = jwt.encode({"user_id": user[0], "exp": datetime.utcnow() + timedelta(minutes=5)}, SECRET_KEY)
            return jsonify({"message": "MFA required", "temp_token": temp_token}), 200
```

- c. Password Reset: Generates a secure token, hashes it, and stores it in `reset_tokens` with an expiration time. Validates the token before allowing password reset.

```
# Password reset request endpoint
@app.route("/request_reset", methods=["POST"])
def request_reset():
    # Generate cryptographically secure random token
    token = secrets.token_urlsafe(32)

    # Hash token before storage (security best practice)
    token_hash = bcrypt.hashpw(token.encode(), bcrypt.gensalt())

    # Set 15-minute expiration for security
    expiration = datetime.now() + timedelta(minutes=15)
```

Data Encryption

- Theoretical Design: The system implements client-side AES encryption before file upload, ensuring servers only handle encrypted data. Encrypted files are stored as BLOBs, while symmetric keys are protected using recipients' public keys for secure sharing. This end-to-end encryption model prevents server-side data exposure.

- Implementation:

- a. File Upload: The client encrypts the file before sending it. The client encrypts the file before sending it.

```
# File upload endpoint
@app.route("/upload", methods=["POST"])
def upload():
    # Prevent directory traversal attacks
    if "../" in file_name:
        return jsonify({"error": "Invalid file name"}), 400

    # Store only encrypted file content (server never sees plaintext)
    encrypted_file = request.files["encrypted_file"].read()
```

- b. File Download: Returns the encrypted file (Base64-encoded) and the encrypted symmetric key. And the client decrypt it locally.

```
# File download endpoint
@app.route("/download/<int:file_id>", methods=["GET"])
def download(file_id):
    # Verify user has permission to access file
    c.execute("SELECT encrypted_symmetric_key FROM permissions WHERE file_id = ? AND user_id = ?",
              (file_id, user_id))

    # Return base64-encoded encrypted data for safe JSON transfer
    return jsonify({
        "encrypted_file": base64.b64encode(file_data[1]).decode()
    })
```

Access Control

- Theoretical Design: Only the file owner can modify/delete files. Shared access requires permissions table entries with encrypted keys.
- Implementation: Enforces file control by verifying the user's ownership before deletion. Then check if the authenticated user matches the file's owner in the database, rejecting the unauthorized requests with a 403 error.

```
# File deletion endpoint
@app.route("/delete_file", methods=["POST"])
def delete_file():
    # Verify requesting user is the file owner
    c.execute("SELECT owner_id FROM files WHERE file_id = ?", (file_id,))
    if not owner or owner[0] != user_id:
        return jsonify({"error": "Not owner"}), 403
```

Log Auditing

- Theoretical Design: Immutable logs record critical events. Admin-only access prevents tampering.
- Implementation: Records who did what and when. This system stores four key details: timestamp, user ID, action type, and context. Database storage prevents tampering. Timestamps enable timeline reconstruction. User IDs ensure accountability.

```
# Logging function - records all critical actions
def log_action(user_id, action, details=None):
    # Store timestamped record of user actions
    c.execute("INSERT INTO logs (timestamp, user_id, action, details) VALUES (?, ?, ?, ?)",
              (datetime.now(), user_id, action, details))
```

Multi-Factor Authentication (MFA) (Extended Functionality)

- Theoretical Design: Users with TOTP secrets must provide a time-based OTP during login.

- Implementation: Validates TOTP. The system checks user-provided codes against server-stored secrets. Successful verification grants a secure JWT token. This adds an essential security layer beyond passwords.

```
# MFA verification endpoint
@app.route("/verify_mfa", methods=["POST"])
def verify_mfa():
    # Verify TOTP code against stored secret
    if pyotp.TOTP(user[0]).verify(totp_code):
        # Generate final auth token after successful MFA
        token = jwt.encode({"user_id": user_id}, SECRET_KEY)
```

Algorithms and Implementation Details → Client Side

User Management

User management is handled through the `register_user`, `login_user`, and `reset_password` functions. During registration, usernames are checked for uniqueness through a database lookup, and passwords are hashed using secure cryptographic algorithms (such as `bcrypt` or `Argon2`) via the `AuthManager` module. When a user logs in, the entered password is verified against the stored hash. If MFA is enabled, the system prompts for a one-time verification code using TOTP (Time-based One-Time Passwords), and the login is only completed upon successful verification. Password reset requires the user to confirm their current password and enter the new one twice, ensuring both intent and correctness.

During registration (`register()`), the username's uniqueness is checked and passwords are securely hashed using **Argon2** (`passlib.hash.argon2`). The user's info is then inserted into the users table with a unique `user_id` (UUID).

In `login()`, credentials are verified against the hash, and if MFA is enabled, the function returns a flag ("mfa_required"). MFA uses a secret managed by the `MFAManager` class and stored in the DB.

Password resets (`reset_password()`) require the correct current password before updating the hash. MFA can be enabled/disabled (`enable_mfa()` / `disable_mfa()`) by verifying a 6-digit TOTP code. The secret is saved to `qr_code.txt` for app scanning.

We've also added the ability to scan a QR code to pop up a six-digit verification code, users need to download the authenticator app (Google) in advance to use this MFA functionality.

Security measures include:

- Secure hashing (Argon2) to protect credentials
- Role-based MFA enforcement
- Parameterized SQL to prevent injection
- UUIDs to avoid predictable identifiers

Password Hashing on Registration

```
password_hash = argon2.hash(password)
cursor.execute(
    "INSERT INTO users (user_id, username, password_hash, mfa_enabled) VALUES (?, ?, ?, ?)",
    (str(uuid.uuid4()), username, password_hash, False)
)
```

MFA Enabled Login Flow

```
if user and argon2.verify(password, user[1]):
    if user[2]:
        return {"status": "mfa_required", "user_id": user[0]}
```

Password Reset with Verification

```
if user and argon2.verify(old_password, user[0]):
    new_hash = argon2.hash(new_password)
    cursor.execute("UPDATE users SET password_hash=? WHERE username=?", ...)
```

Data Encryption

Data encryption is performed exclusively on the client side. When uploading a file, the application uses `file_client.encrypt_file()`, which encrypts the file using the AES-GCM cryptosystem before transmitting it to the server. A unique UUID is generated for each file, and only the encrypted data is stored on the server. The server never has access to the plaintext or the encryption keys. During download, the `decrypt_file()` function is used to reverse the encryption process locally, ensuring the confidentiality of data at rest and in transit. This satisfies the requirement that the server remains a passive adversary incapable of decrypting files.

A unique Fernet-compatible key is derived per user using a combination of the user ID and a secret salt stored in an environment variable. The salt and user ID are concatenated and hashed using **SHA-256**, then encoded to a **URL-safe 32-byte key** (Fernet requirement). This guarantees deterministic, user-specific encryption keys.

The `encrypt_file()` method reads a file's content, encrypts it using **Fernet** (AES in CBC mode + HMAC), generates a UUID as the file ID, and saves the ciphertext to a binary file. The `decrypt_file()` method reverses the process, restoring the original plaintext for download.

This module ensures:

- Client-side encryption (confidentiality from server)
- Per-user encryption keys (derived securely)
- File-level access control via unique UUIDs
- No reliance on static or shared encryption keys

Key Derivation with SHA-256 and Salt

```
salt = os.getenv('ENCRYPTION_SALT', 'default-secret-salt').encode()
digest = hashlib.sha256(self.user_id.encode() + salt).digest()
return base64.urlsafe_b64encode(digest)
```

File Encryption and Secure Storage

```
encrypted_content = self.cipher_suite.encrypt(file_content)
file_id = str(uuid.uuid4())
with open(encrypted_path, 'wb') as f:
    f.write(encrypted_content)
```

File Decryption with Integrity Check

```
decrypted_content = self.cipher_suite.decrypt(encrypted_content)
with open(output_path, 'wb') as f:
    f.write(decrypted_content)
```

Access Control

Access control is enforced at multiple levels. Users may only upload, view, or download files they own or those explicitly shared with them. This is implemented via SQL queries that restrict file access based on the current user's ID. The `share_file` method allows file owners to designate another registered user as a recipient, storing the recipient's ID in the `shared_with` field of the file metadata. Unauthorized access is prevented by ensuring all file retrievals are scoped to files owned by or shared with the logged-in user.

Log Auditing

Log auditing is achieved through a dedicated `AuditLogger` module invoked throughout the application. Key operations such as login, logout, file upload/download, password resets, and file sharing are all logged with timestamps, user IDs, and descriptive action tags. To ensure accountability, log records are immutable once written. Additionally, administrative access to view the entire audit log is granted to the first registered user, treated as the system administrator. Regular users can only view their own activity history, thereby maintaining both privacy and traceability.

On initialization, the logger creates the table (if not present) with fields for log ID (UUID), user ID, action, target, and timestamp. A foreign key constraint links `user_id` to the users table, reinforcing referential integrity.

Logs are written using the `log_action()` method, which inserts timestamped actions into the DB with a secure UUID. Logs can be retrieved via `get_logs()`, either per user or globally (admin only), supporting role-based access.

This module enhances security by enforcing accountability and supports administrator visibility into system activity. It uses:

- `uuid.uuid4()` for secure, unique log IDs
- Parameterized SQL to prevent injection
- `sqlite3` for lightweight storage

Log creation

```
cursor.execute(
    "INSERT INTO audit_logs (log_id, user_id, action, target) VALUES (?, ?, ?, ?)",
    (str(uuid.uuid4()), user_id, action, str(target))
)
```

Role-based log access

```
if user_id:
    cursor.execute("SELECT * FROM audit_logs WHERE user_id=? ORDER BY timestamp DESC LIMIT ?", ...)
```

Multi-Factor Authentication (MFA) (Extended Functionality)

The MFAManager class provides Time-Based One-Time Password (TOTP) support using the pyotp library. It is used by auth.py to enable, verify, and disable MFA for registered users.

It can be used when an authentication application is installed

On initialization, a new secret is generated per user using pyotp.random_base32(). The get_provisioning_uri() method generates a URI for use with mobile authenticator apps (e.g., Google Authenticator), following the standard otpauth:// format. This allows users to scan a QR code and activate MFA easily.

The verify_totp() method checks whether a 6-digit code is valid based on the secret and current time. It also includes debugging outputs for development. This ensures only users with physical access to their MFA device can complete the login.

Security-wise, the implementation:

- Uses standard-compliant TOTP (RFC 6238)
- Avoids reuse of old codes
- Supports a narrow time window (default 30s) for validity

Secret Generation

```
self.secret = pyotp.random_base32()
```

Provisioning URI for QR Code

```
uri = totp.provisioning_uri(name=username, issuer_name="SecureStorageApp")
```

Code Verification

```
totp = pyotp.TOTP(self.secret)
is_valid = totp.verify(code)
```

For each functionality requirement, what your theoretical design is.

To meet the system's core requirements, we designed a secure, client-centric architecture using strong cryptographic standards and database controls. For **user management**, we used argon2 to hash passwords securely and stored MFA secrets per user; logins verify credentials and optionally enforce TOTP-based MFA using pyotp. **File encryption** is handled entirely on the client side using Fernet (symmetric AES encryption), with a per-user key derived from a hashed combination of user_id and a salted environment variable via SHA-256. This ensures that the server never sees plaintext files. For **access control**, each file is assigned a UUID, stored with an owner and optional shared_with user; only matching user IDs can download or decrypt files. Critical events like registration, login, uploads, and sharing are recorded in the audit_logs table with automatic timestamps and UUIDs via the AuditLogger class, enabling **log auditing** and non-repudiation. Admins can view all logs; regular users see only their own. Finally, to ensure **general security**, we use parameterized sqlite3 queries to prevent SQL injection and validate file paths with os.path.basename() to block traversal attacks. Together, these elements form a layered, secure workflow that maintains confidentiality, integrity, and accountability throughout.

To implement your theoretical design, what the technical details are.

To implement our design, we used **Python 3** with supporting libraries: `passlib[argon2]` for password hashing, `pyotp` for TOTP-based MFA, `cryptography`'s Fernet for AES encryption, and `sqlite3` as the lightweight relational database. Each file is encrypted on the client side using a Fernet key derived from the user's ID and a salted secret hashed via `hashlib.sha256`. All encrypted files are stored in a local `encrypted_files/` folder, while file metadata (file ID, owner, shared target) is stored in SQLite. We built a CLI interface in `app_main.py` to interact with users, and `auth.py` manages account creation, login, MFA, and password reset. The `file_client.py` handles secure file processing, and `audit.py` logs all sensitive actions (e.g., upload, login, share) with UUIDs and timestamps. SQL operations are all parameterized to prevent injection. Additionally, we included a minimal Flask API (`main.py`) for optional upload testing. Our database schema is initialized through `init_db.py`, creating the necessary tables with foreign key constraints. The full system is modular, testable, and security-focused.

Library Used

Functionality	Library	Purpose
Password Hashing	<code>bcrypt</code>	Securely hash passwords
Token Generation	<code>Jwt (PyJWT)</code>	Securely hash passwords
MFA (TOTP)	<code>pyotp</code>	Generate/verify time-based OTP codes
Secure Tokens	<code>secrets</code>	Generate cryptographically secure tokens
Database	<code>sqlite3</code>	Store users, files, permissions, logs
Web Framework	<code>Flask</code>	Handle HTTP requests/responses

Technical Challenges - Challenge in TOTP Implementation

During the implementation of TOTP (Time-Based One-Time Password) verification, we initially attempted to generate QR codes dynamically for users to scan. Theoretically, scanning the QR code should display a six-digit verification code. However, while the QR code was successfully generated, the verification code did not appear as expected.

To resolve this issue, we simplified the process by switching to manual code entry. Users now input the verification code directly instead of scanning a QR code. This adjustment not only saved development time but also effectively addressed the problem.

At least 2 Test Cases to verify whether your design can resist attacks.

Based on both the system design and the results from `test_comprehensive.py`, the application demonstrates strong resistance to common security attacks. Passwords are securely hashed with Argon2, and MFA using TOTP adds a second authentication layer. All SQL queries are parameterized, successfully preventing SQL injection attempts. Files are encrypted on the client using Fernet (AES + HMAC), with keys derived securely per user, ensuring the server cannot read plaintext. Access control is enforced by verifying user ownership or sharing permissions, and tests confirm that unauthorized users cannot access protected files. Logs of critical actions are timestamped and tamper-resistant, supporting traceability. The system also sanitizes filenames to block directory traversal. Overall, both theoretical design and practical tests confirm a well-defended, privacy-preserving secure storage system.

`test_edge_cases.py`. blocks duplicate user registration, prevents password resets without valid current credentials, and ensures MFA can't be bypassed with reused or incorrect codes. TOTP codes expire

correctly, preventing replay attacks. Login fails on incorrect passwords regardless of repetition, and SQL injection is mitigated through parameterized queries. These behaviors confirm the system resists brute-force attempts, timing-based MFA exploits, and logic flaws attackers often target at authentication boundaries, making it secure under both normal and edge-case conditions.

Future Work

While our current system meets core functionality requirements, several areas could benefit from further refinement. For example, the file name validation could be strengthened to fully prevent path traversal attacks by implementing stricter checks for special characters. A more efficient file update mechanism might be developed to avoid re-encrypting entire files during minor modifications. The key management system could be enhanced through hardware security modules or periodic key rotation to improve cryptographic security.

We could also try additional security measures like login attempt rate limiting and temporary account lockouts would help protect against brute force attacks. The multi-factor authentication system could be expanded beyond TOTP to include alternative methods like FIDO2 or SMS verification for greater flexibility. The audit logging functionality might be improved by incorporating additional contextual data such as IP addresses to support better forensic analysis.

While the command-line interface provides full functionality, developing a simple graphical or web-based interface could improve accessibility for less technical users. These potential enhancements would build upon the existing foundation while maintaining the system's security-focused design philosophy. Each improvement would address specific limitations while preserving the overall architecture's integrity. The implementation of these features would represent natural progressions of the current work.

Reference

[1] Microsoft, "Threat modeling," Security Development Lifecycle (SDL), 2018. [Online]. Available: <https://www.microsoft.com/en-us/securityengineering/sdl/threatmodeling>